Module 5: Concurrency in Java

Topics

1. What is Concurrent Programming? (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/s3-commentary.html?ou=693191#I)
2. Safety and Liveness (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/s3-commentary.html?ou=693191#II)
3. Designing Active and Passive Objects (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/s3-commentary.html?ou=693191#III)

I. What is Concurrent Programming?

This module discusses concurrent programming, particularly as it relates to the design of objects. But before we begin this discussion, it is first necessary for you to understand what concurrent programming is, and the effects of concurrency on a program.

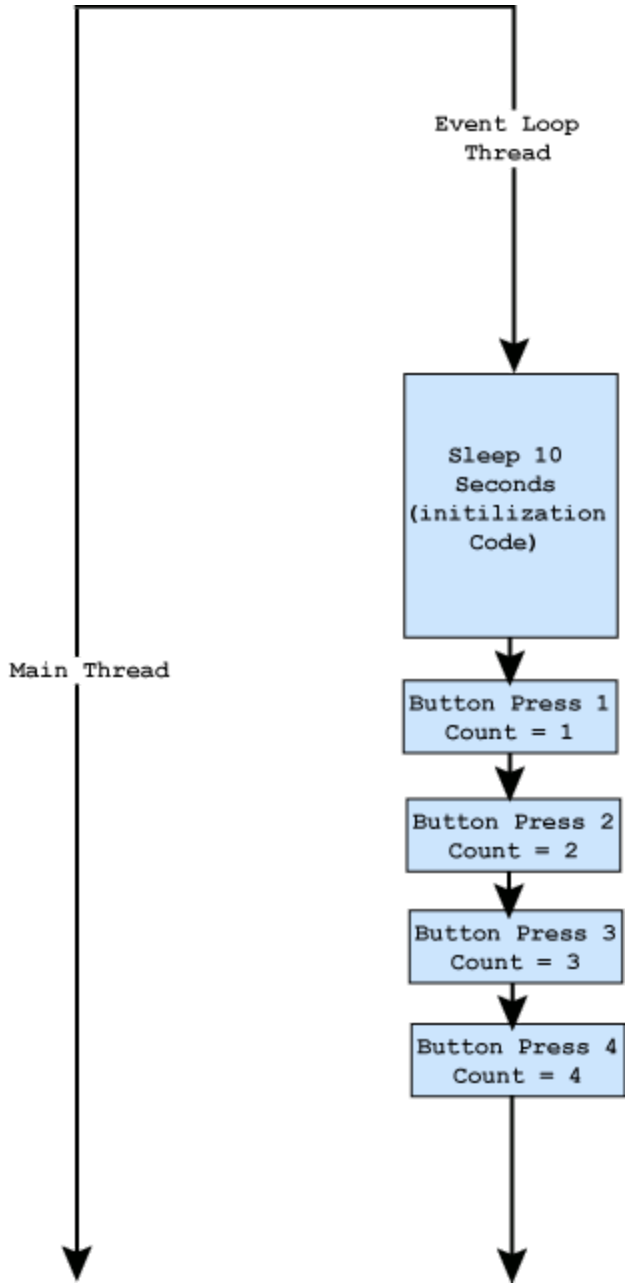
You may know that concurrent programming involves two or more threads of execution occurring at the same time. You may also be aware that concurrent programming involves using Java Threads. You may, however, not know that any program written in Java involves concurrency and Threads (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/popups/popup5-i-1.html?ou=693191) . The concurrency in most Java programs is hidden from the programmer, and generally does not affect a program. That does not mean, though, that the concurrency cannot result in unusual behavior or errors in a program.

To begin the discussion of concurrency, we will examine the reason for studying concurrent programming and give some simple examples of concurrent programs that all students have implemented. Then, we will discuss programming errors specific to concurrent programming, along with methods of finding and fixing these problems. Finally, we will provide an example of how to design a concurrent program using objects.

A. Why Study Concurrent Programming?

Most Java programmers, some with many years of experience, believe that they have never written concurrent programs. In fact, programs in which the programmer explicitly uses threads in her or his program are rare. However, threads are common, and actually nearly universal, in Java programs. To illustrate this point, run the two programs shown below. The first shows a JButton and a variable counter displayed in a JTextField. The first time you press the JBbutton, the program waits 10 seconds, then sets the counter in the JTextField to 1. Subsequently, the counter simply adds one to the value each time you press theJButton, keeping count of the number of times the JButton is pressed. This program is illustrated in figure 5.1. As this figure shows, there are two threads; however, because all the relevant execution is done in the Event Loop thread, the program runs correctly.

Figure 5.1



Button Program with All Logic in Event Loop

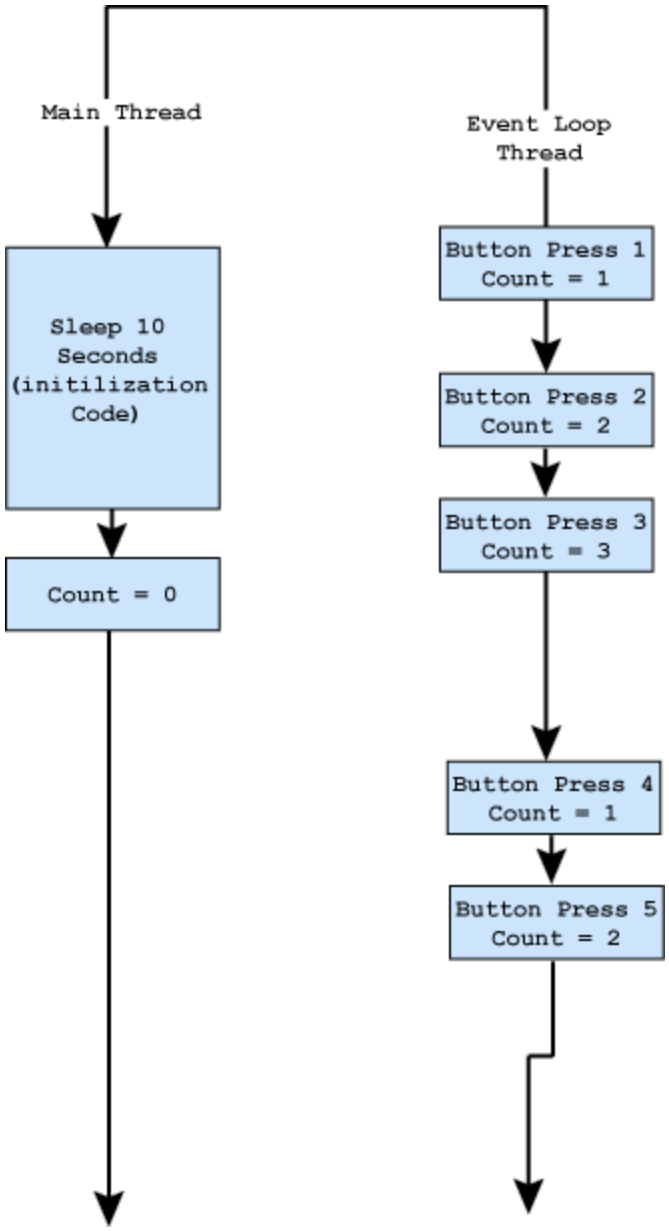
```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class GUIExample1 {
private static int counter = 0;

public static void main(String args[]) {
JFrame frame = new JFrame("Counter");
```

```
Container con = frame.getContentPane();
con.setLayout(new FlowLayout());

con.add(new JLabel("Counter"));
final JTextField counterText = new JTextField("0 ");
con.add(counterText);

JButton counterButton = new JButton("Increment");
counterButton.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
try {
if (counter == 0) {
Thread.sleep(10000);
counter = 1;
}
else {
counter++;
}

counterText.setText("" + counter);
} catch (InterruptedException ie) {
}
}
});
con.add(counterButton);

JButton exitButton = new JButton("Exit");
exitButton.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
System.exit(0);
}
});
con.add(exitButton);

frame.pack();
```

```
frame.setVisible(true);
}
}
```

Contrast this first program with the following program, illustrated in figure 5.2. In this program, the counter is initialized after a 10-second wait (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/popups/popup5-i-2.html?ou=693191) in main(). Note that the call to Thread.sleep() is now in the main() method, not the ActionListener for the JButton. If you press the button after the ten-second delay in main() is finished, the program works correctly. However, if you press the JButton before the ten-second delay is finished, the JButton keeps an accurate count for the first 10 seconds, then resets to 1 and starts over.

Figure 5.2



Button Program with Logic in both Main and Event Loop Threads

```
import java.awt.event.*;

public class GUIExample2 {
private static int counter = 0;

public static void main(String args[]) {
JFrame frame = new JFrame("Counter");
Container con = frame.getContentPane();
con.setLayout(new FlowLayout());
```

```
con.add(new JLabel("Counter"));
final JTextField counterText = new JTextField("0 ");
con.add(counterText);

JButton counterButton = new JButton("Increment");
counterButton.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
counter++;
counterText.setText("" + counter);
}
});
con.add(counterButton);

JButton exitButton = new JButton("Exit");
exitButton.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
System.exit(0);
}
});
con.add(exitButton);

frame.pack();
frame.setVisible(true);

// NOTE: Sleep is moved to main method, which puts
// it in the main thread, not the Event Loop
// which is running the JButton. Note that any
// increments done by the JButton after 10
// seconds are lost.
try {
Thread.sleep(10000);
counter = 1;
} catch (InterruptedException ie) {
```

```
}
}
}
```

What makes these two programs different? The difference comes down to whether the sleep occurs in the same thread of execution as the JButton counter (as in figure 5.1), or in a different thread of execution from the JButton counter (as in figure 5.2). The second program shows the use of threads in that the sleep occurs concurrently with the JButton being pressed. Thus, the second program has at least two threads: one for the main method where the sleep occurs, and one for handling the JButton.

As this second program shows, every program which uses the Java Swing or AWT classes is multithreaded and thus concurrent. If you have written at least one program using Swing or AWT, you have written a concurrent program. Therefore, the first reason to study concurrency is that it appears throughout Java programs. A programmer does not have to explicitly use Threads to have a concurrent program in Java; most concurrency in Java is implicit, and indeed hidden from a programmer.

There is also a second reason to study concurrent programming. As the second example illustrates, not realizing that a program is concurrent and not understanding the implications of concurrency can often result in program errors. These errors can be unpredictable, difficult to reproduce, and hard to fix. For example, consider a situation in which the counter application is used to keep track of the number of people entering a park. Most of the time, the system might function correctly, as people will typically not come to the park as soon as it opens. On certain days, however, the system might not work correctly, such as on a sunny Saturday when a crowd of people shows up at the gate at opening time. This relationship between the days of the week the program is run and the correctness of the program is complex and not obvious. This type of bug would be hard to identify, and, without understanding concurrent programming, difficult to fix.

Although the examples given show that concurrent programming is implicit in GUI programming, concurrent programming does not only occur when you are writing GUI programs using AWT or Swing. There are many other examples where threads are created implicitly, and are thus examples of concurrent programming in Java.

For example, because most Web servers spawn a new thread for each transaction they receive, most programs written on a server to process transactions are concurrent. Databases generally accept queries and updates from multiple sources, so any application that uses a database is concurrent. A program that implements or consumes services in a distributed environment (one that uses multiple computers) is concurrent. Finally, most computer chips today use hyperthreaded (multiple threads in a single cpu) or multicore (multiple CPUs on a single chip) architectures, and to utilize the power, these chips require that applications be written using concurrency.

Because concurrent programming is ubiquitous and can result in incorrect programs, it is important to know how concurrency works, the types of problems it can cause, and how to write correct concurrent programs.

B. Thread-based Programs

Although most concurrency in programs occurs through *hidden* threads created for the programmer (for example, the threads created when using Swing in the examples in the previous section), it is necessary to explicitly create and run threads in Java to understand threading. First, we will show you how to declare and execute java Thread objects. Then, we will show an example comparing a procedural program to a program with multiple threads. The next section of this module will show program errors that are unique to concurrent programs, and how to solve those errors. Finally, we will give a simple example of how to write objects for a concurrent environment using threads and state diagrams.

C. Creating Threads

There are two ways to create a thread in Java. The first involves creating an object that implements a Runnable interface, then passing it to a Thread object's constructor. The thread is then started, and execution begins in the *run()* method of the object passed to the thread. The second way is to extend the class Thread, and add a run() method. The Thread object is then started, and the execution begins in the run() method, as in the previous example. Both of these ways of creating a thread will be covered in the next sections.

Threads and Objects that Implement Runnable

The first, and probably the best, way to create a Thread in a program is to create an instance of an object that implements the Runnable interface. This is illustrated in the following program. In this program, a class MyRunnable is created, and it is given a run() method that simply prints out the message "In run method". When the main() thread starts, an instance of the class MyRunnable is created, and passed to the constructor for the Thread class, creating a Thread object named ct1. The start() method is now called on the Thread object, which in turn begins execution in the run() method of the MyRunnable object. The program then prints out "In run method". The thread ct1 finishes, the main thread finishes, and the program is completed.

```
public class MyRunnable implements Runnable {
public static void main(String[] args) {
MyRunnable ct1 = new MyRunnable();
Thread thread1 = new Thread(ct1);
thread1.start();
}

public void run() {
System.out.println("In run method");
}
}
```

Note the use of the term *begins execution in the run() method*. As will be shown in a following section, the run() in the MyRunnable object is actually executed in a separate thread from the main. It is incorrect to talk about the *start() method calling the run() method*. This implies that the two are run in the same thread (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/popups/popup5-i-3.html?ou=693191) , which is not the case. The run method in the MyRunnable object is executed in a different thread from the thread that is executing in the main method. While the result is the same in this simple program, the difference is important, and will be illustrated in later sections in this module.

In the above program, the identifier myRunnable is only created to be passed to the Thread constructor, so there really is no reason to create an identifier for it. Therefore, the instance of the MyRunnable class can be created directly in the constructor for the Thread class, as illustrated in the following program. These objects can be referred to as anonymous objects.

```
public class MyRunnable implements Runnable {
public static void main(String[] args) {
Thread thread1 = new Thread(new MyRunnable());
thread1.start();
}

public void run() {
System.out.println("In run method");
}
}
```

Following this logic one step further, the Thread identifier is only created so that the start() method can be called on this variable. Once again, there is no need to create an identifier to do this, as it can all be done in a single statement using two anonymous objects, as shown in the following program. The statement now creates the MyRunnable object, passes it to the constructor for the Thread class creating the Thread object, and then directly calls the start() method on this object without creating an identifier. While this may seem confusing at first, the single-statement with anonymous objects approach is much easier to use when writing programs, and will be used extensively throughout this course.

```
public class MyRunnable implements Runnable {
public static void main(String[] args) {
(new Thread(new MyRunnable())).start();
}

public void run() {
```

```
System.out.println("In run method");
}
}
```

Calling the start() method on a Thread object without creating an identifier may cause some confusion. Because there is no identifier, it appears that the Thread object is immediately out of scope, and so can be garbage collected. However, when a Thread object is created, it is registered with the JVM, so that a reference to this object exists even though there is no identifier in the program. Therefore, a Thread object cannot be garbage collected until the thread has completed execution.

Extending class Thread

The second way to create a Thread object is to extend class Thread. Since Thread implements the Runnable interface, and has a run() method defined, this is perfectly valid. And since all methods in Java are virtual, the run() method defined in the sub class will be executed when the start() method is called on a Thread, which is exactly what is expected.

```
public class MyThread extends Thread {
public static void main(String[] args) {
(new MyThread()).start();
}

public void run() {
System.out.println("In run method");
}
}
```

The method of creating an instance of a class that implements Runnable and passing it to the Thread constructor, as demonstrated in the previous section, is actually preferable to the method of creating a Thread object shown above. There are a number of reasons for this:

First, since Java has only single inheritance, extending the Thread class means that the class that is created cannot extend any other class. While there might be no plans to have this subclass ever extend another class, the question is, what is to be gained by extending class Thread instead of using a Runnable object? In this case, extending class Thread costs something, even though it may be small, for no gain.

Second, many libraries—for example, the ThreadPoolExecutor class in Java.util.concurrent—want to use Runnable objects. Although an object that extends Thread would work because Thread implements Runnable, the object would still generate all the extra overhead of the Thread class, which is a very high amount of overhead. This is a potentially large drag on the system.

Finally, a class that extends Thread is typically not a specialized type of Thread, but is related to some entity in the program. For example, you might have a bank with 25 Customer objects that you want to execute as separate threads. In this case, it is not really appropriate to say a Customer is-a Thread. The Customer object is-a Thread only in the limited sense of how it the object is implemented, not in what it means in the problem. Therefore, extending a thread violates the most basic rule of inheritance—namely, that inheritance should represent an "is-a" relationship.

So, unless there is some overarching reason to extend Thread, we recommend that you create a class that implements Runnable, and pass it to the Thread constructor, as illustrated in the previous section.

D. Running Threads

The actual execution of a thread is illustrated by contrasting it with a procedural program that you are probably more familiar with.

When the threads are active and can be executed, the virtual machine (VM) is free to choose any of the threads to execute. Thus, any thread is equally able to execute at any point in time. There are multiple execution paths through the program, with multiple outputs possible. Because each execution path is ordered, these outputs are not totally unordered. But the overall output is only *partially ordered*, as many different orderings are possible depending on how the threads are executed. Thus,

you cannot predict the output from the program. (See Self-Assessment program 2 for an example of how quickly ordered execution paths can result in a large number of different outputs).

E. ThreadPoolExecutor

Creating and starting a thread can cause significant overhead. Although this does not matter for threads that have lifetimes very similar to the lifetime of the program, some applications have threads that start and stop often during a program's lifetime. An example of this would be a Web server wherein each time a transaction is created it is run in a separate thread. Since a Web server runs literally thousands of transactions a second, it is often inefficient to start and stop a thread each time a transaction is run. For these types of systems, Java introduced the ThreadPoolExecutor class.

The ThreadPoolExecutor works just the same as the Thread class, except that it creates a pool of threads that it starts and does not stop. When a transaction needs a thread, it retrieves one from the ThreadPoolExecutor, and uses it as if it were a normal thread. When the transaction has finished using the thread, it simply returns the thread to the ThreadPoolExecutor to be reused.

The ThreadPoolExecutor is only applicable to large, complex systems, and is not useful for the programs created in this class. The important point we want to make is that a method exists to reduce the overhead inherent in using threads.

II. Threats in Concurrent Programming

As we showed in the GUI counter program, there are threats to proper program execution that exist in a concurrent program, but not in a procedural program. These threats arise from the relationships and the timing considerations among threads. There are two types of threats to the proper execution of a concurrent program. The first threat is to program safety. The question to ask regarding program safety is, "if a program finishes, is the answer correct?" The most common problem with program safety is called a *race condition*.

The second threat in concurrent programming is liveness. The question to ask when considering liveness is, "does the program ever finish?" The most common problem with program liveness is called a *deadlock condition*. This section will examine race conditions and deadlock conditions.

A. Race Conditions

Race conditions occur when two threads are contending for the same resource (such as a variable), and perform an operation on that resource that requires more than one machine instruction (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/popups/popup5-i-4.html?ou=693191) . For example, in the following program, a swap operation is performed. In the first step of the swap operation, the thread stores a value in a temporary shared variable, which it plans to retrieve later. If at some point during the execution of that method another thread is able to enter that swap method and change the value, the swap will not give the correct answer.

Note that there is a *race* occurring in this program. If the first thread can complete execution of the swap before the second thread enters and changes the value of the shared variable, the first thread *wins the race*, and the program gives the correct answer. However, if the second thread is able to change the value before the first thread can complete the method, the first thread *loses the race*, and the program gives an incorrect answer.

Race conditions in programs are particularly vexing. They can happen unpredictably, which makes them very difficult to reproduce and find. A race condition might only occur every thousandth time a program is run, which is too frequent for a production application, but too infrequent to allow for easy debugging. Race conditions are also more likely to happen on production systems, where the system (CPU, IO, etc.) utilization is greater than on a lightly used test system, which can make them even more difficult to reproduce in a test environment. It is not unusual for it to take days, weeks, or months to find a race condition, but then take only hours to fix it.

The most basic way to fix a race condition is to lock a section of code before it is entered so that only one thread can run in a critical section at any time.

This type of solution is easy to implement, and handles many of the race conditions that are present in a program. When a system is not amenable to this way of solving a race condition, there are numerous other strategies to try. However, they are all beyond the scope of this class.

B. Deadlock

As mentioned above, deadlock is a situation in which two or more threads share two or more resources, and try to obtain them in such a way as to block the progress of both threads. In a deadlock situation, no thread can finish, and no thread can free up resources. The affected threads sit idle on the computer.

For deadlock to happen, four conditions must hold simultaneously. These are

1. Mutual exclusion. The resources must be nonsharable—that is, only one thread can use a resource at a time. This is true of the Locks class that was used in the previous example; in fact, this Locks class was used to ensure that only one thread could enter a critical section at a time.

2. Hold and wait. At least one of the threads must use two Locks, and must be able to hold one Lock while waiting to acquire the second Lock. This thread can block another thread attempting to acquire the first Lock, while being blocked itself from trying to obtain the second Lock.

3. No preemption. No thread can force any other thread to give up a Lock. A thread can only give up the Lock voluntarily. It typically does this by completing its task and calling unlock (later in this module, we will show how this can be done using Java Condition objects).

4. Circular wait. The threads must acquire the Lock objects in a circular fashion, whereby the first thread holds a Lock needed by a second thread, and the second thread holds a Lock needed by the first thread.

While the example here is for Lock objects, there are many other ways to cause deadlocks. Perhaps the most common deadlock situation in modern computing occurs in databases. It is not uncommon for a series of queries to lock records in the database so that there is a deadlock within the database. The normal solution to this problem is to kill all the queries that are involved in the deadlock, have them roll back the transaction, and start them again after a random interval.

III. Designing Objects for Concurrency

In the last section of the module, we will look at how to design objects to be used in concurrent programs. We provide an example that uses active objects (threads) and passive objects to implement a classic concurrent program known as a producer/consumer (or bounded buffer) problem. The example calls for the following steps:

1. Find the active and passive objects in the problem.
2. Write the active objects. Note that, in Java, active objects can be implemented as threads.
3. Draw a state diagram of the passive object.
4. Implement the state diagram version of the passive object.
5. Write a control object, which creates all the objects in the problem and starts any threads.

Note that this methodology will use Java Condition objects. In order to show you how to implement the code in this section, we will first discuss Condition objects.

A. Condition Variables

The Lock objects previously introduced work well for implementing critical sections. However, it is sometimes necessary to have a program release a Lock object until some specific condition is realized. The mechanism by which this occurs is called a Condition object.

A Condition object is an object that is associated with a Lock object. You can create a Condition object on a Lock object simply by calling the newCondition() method on the Lock object. This creates a Condition object for you (this is another example of a Factory pattern).

A Condition object works by temporarily releasing the associated lock on the Lock object and going into a wait state until the condition it is awaiting is fulfilled. When another thread calls the signal() method on this Condition, the condition is satisfied, and the thread that called the await() method on the Condition object is again ready to run. It must first, however, re-obtain the lock on the associated Lock object.

Be careful when using Condition objects with ReentrantLock objects. Each time you call a ReentrantLock object's lock() method, the call adds one to the number of unlock() calls that must be made before the lock is actually released. However, when you call the await() method on the Condition object, the lock is released, regardless of how many times you called the lock() method.
Thus await() and signal() can be unmatched, but you must match each call to lock() with a call to unlock(). Neglecting this step can cause strange errors if care is not taken, as will be shown in self-assessment question 8.

B. Producer/Consumer Problem

The Producer/Consumer problem involves two or more threads, one or more of which produces data in some fashion, and one or more of which consumes that data in some fashion. In the problem there will be one Producer thread and one Consumer thread. The Producer thread will create 10 integers from 0 to 9. The Consumer thread will take the 10 integers and print them out to the console.

To facilitate the passing of data between the two threads, there is an intermediate object, which is called a *BoundedBuffer*. This object is called a BoundedBuffer because it can store several values from the Producer, thus buffering this data for the Producer so that it can continue to work until the Consumer is ready to take data from the buffer. The BoundedBuffer is bounded

in that it has a limited size—in this case, it can hold two data values. Although the Producer can create more than two data values, it must wait to add data to a full BoundedBuffer until the Consumer has removed one value.

In the producer/consumer problem, the Producer will simply create integers from 1 to 10, and put them to in BoundedBuffer. The Consumer will run until it has taken all 10 values from the BoundedBuffer.

C. Finding the Objects

First, the active and passive objects in the problem are found. Objects in a problem can generally be found by looking for the nouns in the problem. From the description above, the objects needed are the BoundedBuffer, Producer, and Consumer.

In this problem, the Producer and Consumer are active in that they act on the BoundedBuffer, putting data into the BoundedBuffer and taking data from the BoundedBuffer.

The BoundedBuffer is passive in that it takes data from the Producer and gives data to the Consumer. It does nothing on its own; it only responds to requests from the active objects.

D. Writing the Active Objects

Now the active objects are written. When you write an active object, you must create a Thread object to run the object, and code the activities that are to be done by the object in the run() method. In this case, the run() method for the Producer consists of a loop that generates the integer values from 1 to numberOfItems, and adds them to the BoundedBuffer. The run method for the Consumer is a loop from 1 to numberOfItems, which takes the data from the BoundedBuffer and prints it to the console.

The loop itself is relatively easy to implement; however, the problem involves additional steps. It is necessary for the Producer to know how many items to produce, and to have the reference to the BoundedBuffer object that it is passing

data to. These data items cannot be passed into the run() method directly, as the run() method does not take any parameters. Instead, these values are set in the constructor when the object is created. The following is the code for the Producer object:

```
class Producer implements Runnable {
private BoundedBuffer _sb;
public Producer(BoundedBuffer sb) {
_sb = sb;
}

public void run() {
try {
for (int i = 0; i < 10; i++) {
Thread.sleep((int)(Math.random() * 100));
System.out.println("Adding item " + i);
_sb.put(i);
}
} catch(InterruptedException ie) {
}

System.out.println("Producer Finished");
}
}
```

The Consumer object is simply the mirror of the Producer object; it takes the data from the BoundedBuffer and consumes it by printing it out. The following is the code for the Consumer object:

```
class Consumer implements Runnable {
private BoundedBuffer _sb;
public Consumer(BoundedBuffer sb) {
_sb = sb;
}
```

```
public void run() {
try {
for (int i = 0; i < 10; i++) {
Thread.sleep((int)(Math.random() * 1000));
int value = _sb.take();
System.out.println("Got item " + value);
}
} catch(InterruptedException ie) {
}
System.out.println("Consumer Finished");
}
}
```

Now that the Producer and Consumer objects have been written, the next step is to write the passive BoundedBuffer object.

E. Designing the Passive Object

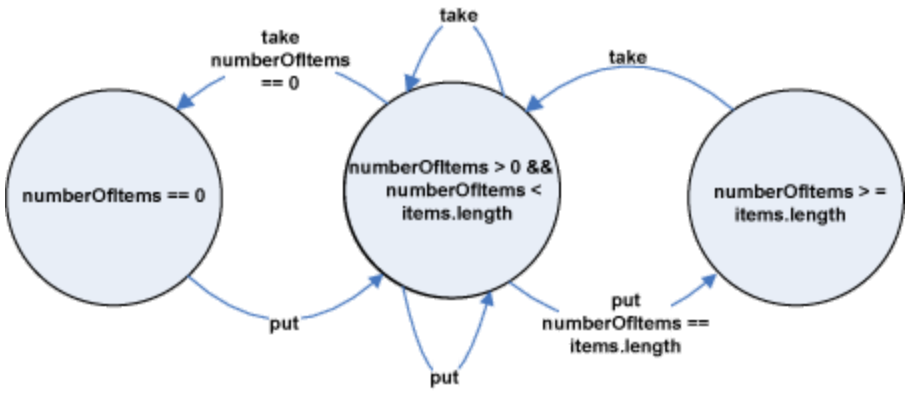
The passive object is more complex to write than the active objects because it does not have a thread of execution to determine what it will do next. Instead, it responds to requests from the active objects.

Passive objects are different than other objects in that you cannot always execute any method in the object. Because these objects are used with threads, the methods must lock and unlock critical sections. Also, the data in the object may preclude a method from running. For example, if the BoundedBuffer is currently full, it cannot process calls to its put() method, so the thread calling the put() method has to wait until a call to the take() method removes an item and makes space in the BoundedBuffer for the item.

To model the BoundedBuffer to only run a method when it is valid to do so, a State Diagram can be used. Figure 5.3 is an example of a State Diagram for the BoundedBuffer. In this diagram, circles represent states. There are three states in which the BoundedBuffer can exist: it can be needEmptySpace (it has no empty

space and cannot receive data); it can be needItem (it has no data, and no object can take data from it); and it can be partiallyFull (it has both data and empty space so that data can be added and taken).

Figure 5.3



State Diagram for the Bounded Buffer

Conditions:
needItem: numberOfItems == 0
needEmptySpace: numberOfItems >= items.length

Methods for the BoundedBuffer are represented as arcs between the circles. If an arc starts from a circle, the method can be run from that state. For example, the put() method can be run from either an needEmptySpace or partiallyFull state. The take() method can be run from either a needItem or partiallyFull state. Note that the needEmptySpace and partiallyFull states are true when the numberOfElements < elements.length(); and the needItem and partiallyFull states are true when the numberOfElements > 0. In the program, these states will be represented with the number of items in the queue.

Also note that Condition objects need to be created to allow the thread to wait for a specific condition. There must be one Condition object that specifies each Condition that a thread can wait on. In this example, the Condition objects needEmptySpace and needItem are created to allow waiting queues for threads which enter the put or take method but that cannot currently execute.

Now that the state diagram is completed, it can be translated into a Java object. In the next section, we show how this is done.

F. Creating the Passive Object

To create the passive object, you would simply translate the state diagram developed in the previous step into a Java class that can be instantiated to create an object. You can do this in four steps:

1. Create the basic outline of the object, using the arcs as the methods for the object. Note at this point that only the methods are declared; the instance variables and method definitions are completed in later steps.

2. Put a precondition into the method that allows the method only to be run when the precondition is true. This precondition represents the state or states that all arcs with the same name (that is, all arcs that are the same method) start from in the state diagram. Note that, to implement these preconditions, you would use Lock objects and Condition objects.

3. Add a postcondition to the method to represent the new state of the passive object. This postcondition represents the state at which the arc ends.

4. Finally, add any processing to implement the object to the method between the pre- and postcondition processing.

Implement the Passive Object Outline

The first step in converting a state diagram into a passive object is to provide the basic outline of the object. In this step, the class for the passive object is defined, as well as the method declarations (only) for this class. For the Producer/Consumer problem, the class is defined and the methods are declared in the class definition below.

Note that, in this method, we have included a Lock object, and that this Lock object is locked when the method is entered and unlocked when this method is exited. The inclusion of this Lock object is necessary because the methods in a passive object will operate on shared data; thus, the methods in a passive object need to be treated as critical sections of code.

Finally, note that the return statement must come after the finally block in this program. Note that this means that any data returned will not be in a critical section, so only local variables should be returned. It will not be safe to return instance or static variables as they will not be guaranteed to be correct because of race conditions.

```
class BoundedBuffer{
private ReentrantLock _bufferLock = new ReentrantLock();

public BoundedBuffer(int size) {
}

public int take() {
try {
_bufferLock.lock();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();

}
return null;
}

public void put(int item) {
try {
_bufferLock.lock();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();

}
}
}
```

Implement the Preconditions

The next step in implementing a passive object is to implement the preconditions the program must check to ensure that the condition to run the method is valid. These conditions are represented by the states in the state diagrams that the arcs (or methods) come from. If the condition to run the method is valid, the execution can continue. If not, the program must wait on a condition variable that represents the condition that must be valid before the method can execute. This is shown for the BoundedBuffer in the following class definition.

Note that the preconditions are checked in a while loop. This is to account for the following race condition: Remember that, when a call to the await() method is made, the underlying lock for the Condition object is dropped. The lock must be reacquired by the thread that has been made ready by the call to the signal() method. However, between the time the thread has been made ready and the time it is able to acquire the lock, another thread can call the lock method on the Lock object. It can thus change the passive object (in this case, the BoundedBuffer) so that the precondition is no longer valid. Thus, every precondition should use a while loop to check whether the precondition is still valid.

```
class BoundedBuffer{
private ReentrantLock _bufferLock = new ReentrantLock();
private Condition _needItem = _bufferLock.newCondition();
private Condition _needEmptySpace = _bufferLock.newCondition();

private int[] _items;
private int _numberOfItems;

public BoundedBuffer(int size) {
_items = new int[size];
}

public int take() {
try {
_bufferLock.lock();
while(_numberOfItems == 0) {
```

```
_needItem.await();
}
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
return null;
}

public void put(int item) {
try {
_bufferLock.lock();
while(_numberOfItems >= _items.length) {
_needEmptySpace.await();
}
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
}
}
```

Implement the Postconditions

The next step is to implement the postcondition processing. The postconditions are represented in the state diagrams and are pointed to by the arcs (or methods). If a program stays in the same state, you know that no thread currently waiting on a condition will be able to run, so there is no need to signal() any Condition objects. However, if the passive object changes state, the program should check to see whether there are any threads waiting on that Condition object. If there are, then a call to the signal() method on that Condition object should be made.

It is not strictly necessary to check to see whether there are threads waiting on the condition because, if no threads are waiting, the call is simply ignored. However, the call to signal could be expensive in some situations, so it is always a good idea

to check for waiting threads. In this case, it would likely incur little penalty to call signal()on the Condition object, but the check for waiting threads is added for illustration purposes.

The source code below shows the modifications to the BoundedBuffer class to add postcondition processing.

```
class BoundedBuffer{
private ReentrantLock _bufferLock = new ReentrantLock();
private Condition _needItem = _bufferLock.newCondition();
private Condition _needEmptySpace = _bufferLock.newCondition();

private int[] _items;
private int _firstItem;
private int _lastItem;
private int _numberOfItems;

public BoundedBuffer(int size) {
_items = new int[size];
}
public int take() {
int returnItem = 0;

try {
_bufferLock.lock();
while(_numberOfItems == 0) {
_needItem.await();
}

if (_bufferLock.hasWaiters(_needEmptySpace))
_needEmptySpace.signal();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
```

```
}
return returnItem;
}

public void put(int item) {
try {
_bufferLock.lock();
while(_numberOfItems >= _items.length) {
_needEmptySpace.await();
}

if (_bufferLock.hasWaiters(_needItem))
_needItem.signal();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
}
}
```

Implement any Processing Needed by the Method

The final step in implementing a passive object is to add any code needed to process the items between the preconditions and postconditions. In the BoundedBuffer class, this is the code needed to manage the buffer. This final, complete, BoundedBuffer class is given in the following source code:

```
class BoundedBuffer{
private ReentrantLock _bufferLock = new ReentrantLock();
private Condition _needItem = _bufferLock.newCondition();
private Condition _needEmptySpace = _bufferLock.newCondition();

private int[] _items;
private int _firstItem;
private int _lastItem;
private int _numberOfItems;
```

```
public BoundedBuffer(int size) {
_numberOfItems = 0;
_items = new int[size];
_firstItem = 0;
_lastItem = 0;
}

public int take() {
int returnItem = 0;

try {
_bufferLock.lock();
while(_numberOfItems == 0) {
_needItem.await();
}

returnItem = _items[_firstItem];
_firstItem = (++_firstItem % _items.length);
_numberOfItems--;

if (_bufferLock.hasWaiters(_needEmptySpace))
_needEmptySpace.signal();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
return returnItem;
}

public void put(int item) {
try {
_bufferLock.lock();
while(_numberOfItems >= _items.length) {
_needEmptySpace.await();
```

```
}

_items[_lastItem] = item;
_lastItem = (++_lastItem % _items.length);
_numberOfItems++;

if (_bufferLock.hasWaiters(_needItem))
_needItem.signal();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
}
}
```

G. Control Objects

Now that you have created all the active and passive objects, the final step is to put them together to generate a program. This is done by creating the passive object(s) needed to coordinate the active objects, then creating and starting the active objects. The following object accomplishes this goal for
the Producer/Consumer problem:

```
public class ProducerConsumer {
public static void main(String[] args) {
BoundedBuffer sb = new BoundedBuffer(2);
(new Thread(new Producer(sb))).start();
(new Thread(new Consumer(sb))).start();
}
}
```